Azure & Azure DevOps — Interview Ready Answers

Comprehensive answers with examples for all 14 Azure & Azure DevOps questions

14 Questions • Azure DevOps • AKS • Key Vault • Blob Storage • Azure AD • Pipelines • Terraform on Azure

---

Q1. How would you architect a secure, auditable CI/CD pipeline in Azure DevOps for multi-environment (Dev/Test/Prod) deployments?

Answer

A production-grade Azure DevOps pipeline for multi-environment deployments combines YAML pipelines, environment approvals, branch policies, and full audit logging across every stage.

High-Level Architecture

```
Feature Branch → PR → main branch
  Stage 1: Build & Test    (runs on every PR and push)
  Stage 2: Deploy Dev      (auto on merge to main)
  Stage 3: Deploy Test     (auto after Dev passes)
  Stage 4: Deploy Prod     (manual approval gate required)
```

Pipeline YAML Structure

```
trigger:
  branches:
    include: [main]

stages:
- stage: Build
  jobs:
  - job: BuildAndTest
    steps:
    - task: Docker@2
      inputs:
        command: buildAndPush
        containerRegistry: myACR
        tags: $(Build.SourceVersion)

- stage: DeployDev
  dependsOn: Build
  jobs:
  - deployment: DeployToDev
    environment: dev          # Azure DevOps Environment
    strategy:
      runOnce:
        deploy:
          steps:
          - task: HelmDeploy@0
            inputs:
              azureSubscriptionEndpoint: dev-service-connection
              command: upgrade
              chartName: ./charts/myapp
              releaseName: myapp
              overrideValues: image.tag=$(Build.SourceVersion)

- stage: DeployProd
  dependsOn: DeployTest
  jobs:
  - deployment: DeployToProd
    environment: prod          # Has approval gate configured
    strategy:
      runOnce:
        deploy:
          steps:
          - task: HelmDeploy@0
```

Security Controls

- Service Connections with Workload Identity Federation — No passwords or client secrets stored. The pipeline authenticates to Azure using OIDC federation. Each environment has its own scoped service connection.
- Branch Policies on main — Require PR reviews (minimum 2 reviewers), require passing CI checks, disallow force pushes, require linked work items.
- Azure DevOps Environments — Each environment (dev/test/prod) is a named environment in Azure DevOps with approval gates, checks, and deployment history.
- Prod Approval Gate — Configured on the prod environment: require manual approval from two senior engineers before deployment proceeds.
- Secret Variables — All secrets stored in Azure Key Vault. Pipeline references secrets via Key Vault variable groups — never hardcoded.

Auditability

- Every pipeline run is recorded in Azure DevOps with: who triggered it, what commit, which stage, who approved, what duration.
- Azure Monitor + Diagnostic Settings — Export pipeline logs to Log Analytics workspace for retention and compliance querying.
- Immutable image tags — Docker images tagged with the Git commit SHA. Every deployment is traceable to a specific commit.
- Environment deployment history in Azure DevOps shows every deployment: time, triggering pipeline, commit, approver.

Per-Environment Configuration

Setting	Dev
Approval gate	None
Service connection	dev-sub
Secret source	Key Vault dev/
Deployment strategy	Rolling

> *Tip: Use Deployment Jobs (deployment:) rather than regular jobs (job:) for all deployment stages. Deployment Jobs unlock Azure Environments, approval gates, deployment history, and rollback tracking.*

Q2. Differentiate between classic pipelines and YAML pipelines in Azure DevOps. Which one is better for Infrastructure-as-Code and why?

Answer

Azure DevOps offers two pipeline authoring models. Classic (GUI-based) pipelines were the original approach, while YAML pipelines are the modern, code-first model recommended for all new projects.

Feature Comparison

Feature	Classic Pipeline	YAML Pipeline
Authoring	GUI drag-and-drop editor	Code in .yml file alongside source code
Version control	Not in source code (stored in Azure DevOps)	Fully version-controlled in Git — every change is a commit
Code review	No — changes apply immediately	Yes — pipeline changes go through PR review
Reusability	Limited — copy-paste stages	Templates (extends:, template:) — DRY, composable
Multi-stage	Limited support	First-class — stages, jobs, steps natively supported
Audit trail	Basic run history	Full Git history of every pipeline change
Branch-specific behaviour	Difficult	Natively supports trigger conditions, branch filters

IaC suitability	Poor	Excellent
Portability	Tied to Azure DevOps	YAML is portable; can be adapted for GitHub Actions
Secrets	Variable groups (manual)	Variable groups + Key Vault integration + OIDC

Why YAML Pipelines Are Better for IaC

- Pipeline-as-Code: When your pipeline provisions infrastructure with Terraform, the pipeline itself should also be code — reviewed, tested, versioned, and auditable. A GUI-configured pipeline managing IaC is a contradiction.
- PR-based change control: All pipeline changes go through a code review. A junior engineer cannot accidentally change the production Terraform apply stage without a reviewer approving the PR.
- Template reusability: Define a Terraform apply template once and reuse it across 10 environments. Classic pipelines require copy-pasting stages, leading to drift.
- Conditional execution: YAML conditions make it trivial to run terraform plan on PRs and terraform apply only on merges to main:

```
- script: terraform plan
  condition: eq(variables['Build.Reason'], 'PullRequest')
- script: terraform apply -auto-approve
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
```

- Branch-specific behaviour: Run different Terraform workspaces or variable files per branch/environment without complex workarounds.

Real-World YAML IaC Pipeline Example

```
stages:
- stage: TerraformPlan
  jobs:
  - job: Plan
    steps:
    - task: TerraformTaskV4@4
      inputs:
        command: plan
        workingDirectory: $(System.DefaultWorkingDirectory)/terraform
        environmentServiceNameAzureRM: dev-service-connection
        backendServiceArm: terraform-state-connection
        backendAzureRmResourceGroupName: rg-terraform-state
        backendAzureRmStorageAccountName: tfstateaccount
        backendAzureRmContainerName: tfstate
        backendAzureRmKey: dev/terraform.tfstate
```

When to Still Use Classic Pipelines

Classic pipelines are acceptable only for legacy projects already using them, or for teams with non-technical stakeholders who must configure pipelines. All new IaC and application pipelines should use YAML.

> *Tip: Migrate existing Classic pipelines to YAML using the 'Export to YAML' option in Azure DevOps. It's not perfect but provides a starting point. Complete the migration before the pipeline grows more complex.*

Q3. What rollback strategies have you implemented in mission-critical environments? Can you detail a real-world example?

Answer

Rollback strategy depends on the deployment type — Kubernetes, Azure App Service, infrastructure (IaC), or database. A mission-critical system needs rollback at every layer.

Strategy 1 — Helm Rollback (Kubernetes / AKS)

Helm maintains a release history. If a deployment fails health checks or triggers alerts, rollback is a single command:

```
# Check Helm release history
helm history myapp -n production
# REVISION  STATUS     CHART         DESCRIPTION
# 14        deployed   myapp-1.3.2   Upgrade complete
# 15        failed     myapp-1.3.3   Upgrade failed

# Rollback to previous revision
helm rollback myapp 14 -n production
```

In Azure DevOps pipelines, automate rollback by running helm rollback if the post-deploy smoke test fails:

```
- script: |
    helm upgrade myapp ./chart --atomic --timeout 5m || \
    helm rollback myapp --wait
  displayName: Deploy with auto-rollback on failure
```

The --atomic flag makes Helm automatically rollback if the upgrade fails — the release never enters a broken state.

Strategy 2 — Azure Pipelines Re-run Previous Release

In Azure DevOps Release Pipelines or YAML pipelines, re-trigger the previous successful pipeline run pointing to the last known-good image tag. Since images are tagged with immutable Git SHAs, deploying SHA abc1234 is always reproducible.

Strategy 3 — Blue-Green Instant Rollback

In a blue-green setup, rollback means switching traffic back to the old environment immediately — seconds, not minutes:

```
# Rollback: point the AKS service selector back to blue pods
kubectl patch service myapp-service \
  -p '{"spec":{"selector":{"version":"blue"}}}'
```

Strategy 4 — Terraform State Rollback

For IaC rollbacks, revert the Git commit that changed the Terraform code, then run terraform apply. The previous state is restored. For partial failures:

```
# Revert to a specific state version stored in Azure Blob
az storage blob download \
  --container-name tfstate \
  --name prod/terraform.tfstate.backup \
  --file terraform.tfstate
```

Real-World Example — Fintech Payment Service Rollback

Situation: A new release of the payment microservice caused a 3% increase in transaction failures at 2 AM. The post-deployment monitoring alert triggered in PagerDuty.

- T+0: PagerDuty alert fires. On-call engineer acknowledges.
- T+3min: Engineer identifies the failing deployment via Azure Monitor Application Insights — error rate spiked after the 1:55 AM deployment.
- T+5min: Helm rollback executed: helm rollback payments 22 -n production
- T+7min: Error rate returns to baseline. Service restored. Incident severity downgraded.
- T+8min: Rollback pipeline stage re-tagged the previous image as 'stable' and updated the deployment record in Azure DevOps.
- T+24hrs: Root cause analysis completed. The new release had a race condition in the payment validation logic. Fixed in a patch release the next day.

Database Rollback — The Hard Part

- Application rollback does NOT rollback database schema changes. Use the Expand-Contract migration pattern: additive changes only (add column, add table). Never drop columns in the same release as the code change.
- Use migration tools (Flyway, Liquibase) with down migration scripts for emergency database rollbacks.

> *Tip: Define a rollback SLA (e.g., production rollback must complete within 10 minutes). Practice it in quarterly fire drills so it becomes muscle memory, not a crisis scramble.*

Q4. Explain the use of Jenkins Shared Libraries or Azure DevOps Templates for enterprise-level DRY (Don't Repeat Yourself) pipelines.

Answer

In large organisations with many teams and repositories, duplicating pipeline code across repos creates maintenance nightmares. Azure DevOps Templates (and Jenkins Shared Libraries) solve this by centralising reusable pipeline logic in a dedicated repository.

Azure DevOps Pipeline Templates

Templates allow you to define reusable steps, jobs, or stages in a central template repository, then reference them from any pipeline in the organisation.

Template Repository Structure

```
pipeline-templates/                 # Dedicated shared repo
├── steps/
│   ├── docker-build.yml
│   ├── terraform-plan.yml
│   ├── terraform-apply.yml
```

```
│   ├── helm-deploy.yml
│   └── security-scan.yml
├── jobs/
│   ├── build-job.yml
│   └── deploy-job.yml
└── stages/
    ├── ci-stage.yml
    └── deploy-stage.yml
```

Template Definition — steps/helm-deploy.yml

```
parameters:
  - name: environment
    type: string
  - name: imageTag
    type: string
  - name: serviceConnection
    type: string

steps:
- task: HelmDeploy@0
  displayName: 'Deploy ${{ parameters.environment }}'
  inputs:
    azureSubscriptionEndpoint: ${{ parameters.serviceConnection }}
    command: upgrade
    releaseName: myapp-${{ parameters.environment }}
    overrideValues: image.tag=${{ parameters.imageTag }}
- script: |
    echo 'Running smoke tests...'
    curl -f https://${{ parameters.environment }}.myapp.com/health
  displayName: Post-deploy health check
```

Consuming the Template in Application Pipelines

```
# In the application's azure-pipelines.yml
resources:
  repositories:
  - repository: templates
    type: git
    name: platform/pipeline-templates
    ref: refs/tags/v1.5.0    # Pin to a specific version

stages:
- stage: DeployDev
  jobs:
  - job: Deploy
    steps:
    - template: steps/helm-deploy.yml@templates
      parameters:
        environment: dev
        imageTag: $(Build.SourceVersion)
        serviceConnection: dev-azure-connection
```

Benefits of This Approach

- Single source of truth — Fix a bug in the deploy template once, all 50 application pipelines benefit immediately.
- Consistency — Every application goes through identical security scanning, testing, and deployment steps. No team can skip security checks.
- Version pinning — Teams pin to a specific template version (v1.5.0) and upgrade deliberately. Breaking changes don't surprise teams.
- Separation of concerns — Platform team owns the templates (HOW to deploy). Application teams own what they deploy (WHAT to deploy).

Jenkins Shared Libraries (Equivalent)

Jenkins Shared Libraries achieve the same goal. Stored in a separate Git repo, they provide reusable Groovy functions and pipeline steps:

```
// vars/helmDeploy.groovy in shared library
def call(Map config) {
  sh """
    helm upgrade ${config.release} ${config.chart} \\
      --set image.tag=${config.imageTag} \\
      --namespace ${config.namespace}
  """
}
```

```
// Jenkinsfile in application repo
@Library('platform-shared-lib@v1.5') _
helmDeploy(release: 'myapp', chart: './charts', imageTag: env.GIT_COMMIT, namespace:
'prod')
```

Tip: Version your template repository with Git tags and require teams to reference a specific tag, not 'main'. This prevents a platform team change from accidentally breaking all application pipelines simultaneously.

Q5. How do you handle multi-repo CI/CD pipelines when deploying a microservices-based application with interdependencies?

Answer

In a microservices architecture, each service lives in its own repository with its own pipeline. Managing dependencies between services — versioning, coordinated deployments, and integration testing — requires a deliberate strategy.

Common Challenges

- Service A's new feature depends on a breaking change in Service B's API.
- Running integration tests requires multiple services running simultaneously.
- You want to deploy services independently but certain releases must be coordinated.

Strategy 1 — Semantic Versioning + Contract Testing

Each service publishes versioned APIs and container images. Consumer-Driven Contract Testing (using Pact) ensures Service A's expectations of Service B's API are always met before deployment:

- Service B publishes a Pact contract when its API changes.
- Service A's pipeline verifies it can work with the new contract before Service B is deployed.
- If contract breaks, Service B's pipeline fails — preventing an incompatible release.

Strategy 2 — Monorepo-Style Coordination via a Deployment Repository

Maintain a dedicated deployment repository that holds the Helm chart or Kustomize overlays for the entire application. Service repos push new image tags to this repo via PRs, and the deployment repo's pipeline orchestrates the full rollout:

```
microservices-deployment/          # Central deployment repo
├── charts/
│   ├── auth-service/values.yaml   # image.tag: abc1234
│   ├── order-service/values.yaml  # image.tag: def5678
│   └── payment-service/values.yaml
└── azure-pipelines.yml            # Deploys all or changed services
```

Strategy 3 — Trigger Downstream Pipelines

Service A's pipeline, after a successful deployment to staging, triggers Service B's integration test pipeline using Azure DevOps pipeline triggers or REST API calls:

```
# azure-pipelines.yml in Service A
- stage: TriggerIntegrationTests
  jobs:
  - job: Trigger
    steps:
    - task: TriggerBuild@3
      inputs:
        definitionId: 42           # Integration test pipeline ID
        buildParameters: '{"serviceATag": "$(Build.SourceVersion)"}'
```

Strategy 4 — Environment Promotion Gates

Each service deploys independently to a shared integration environment. A separate gate pipeline checks that all services in the integration environment are healthy and compatible before promoting to production.

```
Integration Environment Health Gate:
  ✓ auth-service v1.4.2 — healthy
  ✓ order-service v2.1.0 — healthy
  ✓ payment-service v3.0.1 — healthy
  → All green: Promote all services to production
```

Dependency Management Best Practices

- Shared API contracts — Use OpenAPI specs stored in a shared schema registry. Services validate against the schema in their CI pipelines.
- Backward compatibility — New API versions must be backward compatible for at least 2 release cycles. Never remove a field without deprecating it first.

- Feature flags — Deploy code changes behind feature flags. Enable interdependent features simultaneously across services without coordinating deployments.
- Service mesh (Istio/Linkerd) — Traffic management at the network level for canary rollouts of multiple services simultaneously.

> *Tip: Avoid deployment synchronisation between services as much as possible. The ability to deploy services independently is the primary benefit of microservices. Use API versioning and backward compatibility to keep services decoupled.*

Q6. Describe techniques to cache Docker layers or NuGet/NPM packages in pipelines for speed optimization.

Answer

Build caching is one of the highest-impact optimizations in CI/CD pipelines. Properly configured caching can reduce build times by 60-80%, cutting compute costs and improving developer feedback loops.

1. Docker Layer Caching

Docker builds images in layers. If a layer's input hasn't changed, Docker reuses the cached layer instead of rebuilding it. The key is ordering your Dockerfile instructions correctly — infrequently changing instructions first:

```
# Bad ordering — dependencies reinstalled on every code change
COPY . .
RUN npm install

# Good ordering — dependencies cached separately from code
COPY package.json package-lock.json ./
RUN npm install              # ← Only re-runs when package.json changes
COPY . .                    # ← Code changes don't invalidate npm cache
RUN npm run build
```

Docker Layer Cache in Azure Pipelines

Azure Pipelines agents don't persist Docker cache between runs by default. Use BuildKit inline cache with ACR:

```
- task: Docker@2
  inputs:
    command: build
    Dockerfile: Dockerfile
    buildContext: .
    arguments: |
      --cache-from $(acrName).azurecr.io/myapp:cache
      --build-arg BUILDKIT_INLINE_CACHE=1
    tags: |
      $(acrName).azurecr.io/myapp:$(Build.SourceVersion)
      $(acrName).azurecr.io/myapp:cache
  env:
    DOCKER_BUILDKIT: 1
```

2. NPM / Node.js Package Caching

Cache the node_modules directory or npm cache between pipeline runs using the Azure Pipelines Cache task:

```
- task: Cache@2
  inputs:
    key: 'npm | $(Agent.OS) | package-lock.json'
    restoreKeys: |
      npm | $(Agent.OS)
    path: $(npm_config_cache)
  displayName: Cache NPM packages

- script: npm ci
  displayName: Install dependencies (uses cache if available)
```

3. NuGet Package Caching (.NET)

```
- task: Cache@2
  inputs:
    key: 'nuget | $(Agent.OS) | **/packages.lock.json,!**/bin/**,!**/obj/**'
    restoreKeys: |
      nuget | $(Agent.OS)
    path: $(NUGET_PACKAGES)
  displayName: Cache NuGet packages

- task: DotNetCoreCLI@2
```

```
  inputs:
    command: restore
    feedsToUse: select
```

4. pip / Python Package Caching

```
- task: Cache@2
  inputs:
    key: 'pip | $(Agent.OS) | requirements.txt'
    path: $(PIP_CACHE_DIR)
  displayName: Cache pip packages

- script: pip install -r requirements.txt
```

5. Maven / Gradle Caching (Java)

```
- task: Cache@2
  inputs:
    key: 'maven | $(Agent.OS) | **/pom.xml'
    path: $(MAVEN_CACHE_FOLDER)
```

6. Self-Hosted Agents with Persistent Cache

Self-hosted Azure DevOps agents (EC2, Azure VM, AKS) persist the workspace and Docker layer cache between builds naturally — no Cache task needed. This is the most effective approach for high-frequency build teams.

Technique	Typical Time Saved
Dockerfile layer ordering	20-40%
NPM/pip cache task	30-50%
Docker BuildKit + registry cache	40-60%
Self-hosted agents	50-70%
All combined	60-80%+

> *Tip: Always use npm ci instead of npm install in CI pipelines. It installs exactly what's in the lockfile (reproducible), fails on lockfile mismatch, and is significantly faster in clean environments.*

Q7. How do you integrate Azure Key Vault with DevOps pipelines securely, especially with rotating secrets or certificates?

Answer

Azure Key Vault is the authoritative secret store for Azure environments. Integrating it with Azure Pipelines eliminates hardcoded secrets and allows centralized rotation without pipeline changes.

Integration Method 1 — Key Vault Linked Variable Groups (Recommended)

Create a Variable Group in Azure DevOps linked to an Azure Key Vault. Secrets are fetched at pipeline runtime — the pipeline never stores them:

- Step 1: In Azure DevOps → Library → Variable Groups → Link secrets from Azure Key Vault.
- Step 2: Select the Key Vault, subscription, and the specific secrets to expose.
- Step 3: Grant the Azure DevOps service connection's managed identity Get and List permissions on the Key Vault.
- Step 4: Reference the variable group in your pipeline:

```
variables:
- group: prod-keyvault-secrets   # Linked to Key Vault

steps:
- script: |
    echo 'Connecting to DB...'
    ./deploy.sh --db-password $(db-password)  # db-password from Key Vault
  env:
    DB_PASSWORD: $(db-password)               # Masked in logs automatically
```

Integration Method 2 — AzureKeyVault Task

Fetch secrets dynamically during pipeline execution using the AzureKeyVault task:

```
- task: AzureKeyVault@2
  inputs:
    azureSubscription: prod-service-connection
```

```
    KeyVaultName: kv-prod-secrets
    SecretsFilter: 'db-password,api-key,ssl-certificate'
    RunAsPreJob: true

# Secrets are now available as pipeline variables
- script: echo '$(db-password)' | docker login --password-stdin
```

Handling Rotating Secrets

When a secret rotates in Key Vault (manually or via auto-rotation), the pipeline picks up the new value on the next run automatically — no pipeline changes needed. This is the key advantage over storing secrets as pipeline variables.

- For automatic rotation: Key Vault supports auto-rotation for Azure Storage keys and SQL passwords via rotation policies. Enable rotation for 30, 60, or 90-day cycles.
- For certificate rotation: Store certificates as Key Vault certificate objects (not secrets). Auto-renewal integrates with DigiCert or Let's Encrypt directly.

Using Certificates from Key Vault in AKS

The Secrets Store CSI Driver (with Azure Key Vault provider) mounts Key Vault secrets and certificates directly as files in pods — no manual certificate management:

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-keyvault-certs
spec:
  provider: azure
  parameters:
    usePodIdentity: 'false'
    clientID: '<managed-identity-client-id>'
    keyvaultName: kv-prod-secrets
    objects: |
      array:
        - |
          objectName: tls-certificate
          objectType: cert
    tenantID: '<tenant-id>'
```

Security Best Practices

- Use Managed Identity (not client secret) for the service connection to Key Vault — eliminates the chicken-and-egg problem of storing credentials to retrieve credentials.
- Apply Key Vault access policies with least privilege: pipelines need only Get/List on secrets. Key administrators need Set/Delete.
- Enable Key Vault soft delete and purge protection — prevents accidental deletion of secrets.
- Enable Key Vault diagnostic logging — audit every secret access in Log Analytics.
- Separate Key Vaults per environment — prod secrets are physically isolated from dev/test.

> *Tip: Never print or echo secret values in pipeline scripts — Azure DevOps automatically masks known secret values in logs, but only if they're declared as secret variables. Raw echoing can bypass masking.*

Q8. What is the purpose of manual intervention gates, and how would you enforce them before production release?

Answer

Manual intervention gates (also called approval gates or quality gates) are checkpoints in a deployment pipeline that pause execution and require human sign-off before proceeding. They are essential for controlling what reaches production-critical environments.

Why Manual Gates Are Needed

- Compliance and audit requirements — Regulated industries (banking, healthcare) require documented human approval of every production change.
- Risk management — Automated tests don't catch everything. A human reviewer can assess business risk context that tests cannot.
- Coordination — Some deployments require coordination with external teams, customers, or maintenance windows.
- Emergency brake — Gives the team a chance to pause if monitoring shows problems in the previous stage.

Implementing Approval Gates in Azure DevOps — Method 1: Environment Approvals

This is the recommended approach for YAML pipelines. Configure the 'prod' Environment with required approvers:

- In Azure DevOps → Environments → prod → Approvals and checks → Approvals.
- Add required approvers (specific users or groups).
- Set: 'All approvers must approve' or 'any N of M approvers'.
- Set timeout: If not approved within 8 hours, the deployment expires.

The pipeline pauses automatically when it reaches the production deployment job and sends notifications to approvers:

```
- stage: DeployProd
  dependsOn: DeployStaging
  jobs:
  - deployment: DeployToProd
    environment: prod        # ← Approval gate is configured on this environment
    strategy:
      runOnce:
        deploy:
          steps:
          - template: steps/helm-deploy.yml@templates
```

Method 2: Manual Validation Task

For more flexibility, use the ManualValidation task inside a pipeline job to pause and require manual sign-off with comments:

```
- stage: PreProdValidation
  jobs:
  - job: WaitForApproval
    pool: server              # Server job — no agent needed
    steps:
    - task: ManualValidation@0
      inputs:
        notifyUsers: |
          engineering-lead@company.com
          change-board@company.com
        instructions: |
          Review the staging deployment:
          - Check metrics dashboard: https://grafana.company.com
          - Verify smoke tests passed
          - Confirm change ticket CAB-1234 is approved
          Approve to proceed or reject to abort.
        onTimeout: reject
        timeoutInMinutes: 480   # 8-hour window
```

Method 3: Quality Gate Checks on Environments

Beyond human approval, add automated pre-deployment checks to the environment:

- Business hours check — Only allow deployments between 09:00 and 17:00 on weekdays.
- Query Azure Monitor — Fail gate if current error rate on staging exceeds threshold.
- Invoke Azure Function — Custom logic (check change management system, compliance database).

Change Management Integration

For enterprises using ServiceNow or Jira Service Management, integrate the gate with the change management system:

- Gate checks that a Change Request (CR) ticket exists and is in 'Approved' status.
- Gate records the deployment details back to the CR automatically after deployment.
- This creates a fully auditable change record: who requested, who approved, what was deployed, when.

> *Tip: Configure gates to notify via Microsoft Teams or email with direct approve/reject links. Reviewers can approve without opening the Azure DevOps UI — reduces approval latency from hours to minutes.*

Q9. How do you implement compliance as code (security scanning, code coverage, policy checks) in Azure DevOps pipelines?

Answer

Compliance as Code embeds security, quality, and policy checks directly into the pipeline — making compliance a continuous, automated process rather than a periodic manual audit.

1. Static Application Security Testing (SAST)

Scan source code for security vulnerabilities before the application is built:

```
- task: Mend Bolt@0             # Formerly WhiteSource — open source vulnerabilities
  displayName: SCA - Open Source Scan
```

```
- script: |
    pip install semgrep
    semgrep --config=auto --error ./src   # SAST — code patterns
  displayName: SAST - Semgrep

- task: SonarCloudPrepare@1     # SonarCloud code quality
  inputs:
    SonarCloud: sonarcloud-connection
    organization: my-org
    scannerMode: CLI
    configMode: manual
    cliProjectKey: myapp
```

2. Container Image Scanning

Scan the built Docker image for OS and dependency CVEs before pushing to the registry:

```
- script: |
    trivy image \
      --exit-code 1 \
      --severity CRITICAL,HIGH \
      --format sarif \
      --output trivy-results.sarif \
      $(acrName).azurecr.io/myapp:$(Build.SourceVersion)
  displayName: Container Security Scan - Trivy

- task: PublishBuildArtifacts@1
  inputs:
    artifactName: trivy-sarif
    pathToPublish: trivy-results.sarif
```

3. Infrastructure as Code Scanning

Scan Terraform, ARM templates, and Kubernetes manifests for misconfigurations:

```
- script: |
    # Checkov — IaC security scanner
    pip install checkov
    checkov -d ./terraform --framework terraform \
      --output junitxml > checkov-results.xml
  displayName: IaC Security Scan - Checkov

- script: |
    # tfsec — Terraform-specific
    tfsec ./terraform --format junit > tfsec-results.xml
  displayName: Terraform Security - tfsec
```

4. Code Coverage Enforcement

Fail the pipeline if code coverage drops below the minimum threshold:

```
- task: DotNetCoreCLI@2
  inputs:
    command: test
    arguments: '--collect "Code Coverage" --settings coverlet.runsettings'

- task: PublishCodeCoverageResults@1
  inputs:
    codeCoverageTool: Cobertura
    summaryFileLocation: '**/coverage.xml'
    failIfCoverageEmpty: true

- script: |
    COVERAGE=$(grep -oP 'line-rate="\K[0-9.]+' coverage.xml | head -1)
    if (( $(echo "$COVERAGE < 0.80" | bc -l) )); then
      echo 'Coverage below 80% threshold. Failing build.'
      exit 1
    fi
  displayName: Enforce 80% code coverage
```

5. Policy Checks — OPA / Gatekeeper

Use Open Policy Agent (OPA) to enforce organisational policies — e.g., all Kubernetes manifests must have resource limits, all containers must run as non-root:

```
- script: |
    conftest test ./k8s/ --policy ./policies/
  displayName: OPA Policy Check - conftest
```

6. Secret Detection

Detect accidentally committed secrets (API keys, passwords) before they reach the repository:

```
- script: |
    git secrets --scan
    trufflehog git file://. --json > secrets-scan.json
    if [ -s secrets-scan.json ]; then exit 1; fi
  displayName: Secret Detection - TruffleHog
```

Publishing Results to Azure DevOps

Publish all test results and security findings to Azure DevOps for visibility and tracking:

```
- task: PublishTestResults@2
  inputs:
    testResultsFormat: JUnit
    testResultsFiles: '**/checkov-results.xml'
    testRunTitle: IaC Security Checks
```

> *Tip: Add quality gates using branch policies: require that the 'Compliance Checks' pipeline passes before any PR can be merged. This makes security a blocking requirement, not an optional step.*

Q10. How do you enforce pipeline security — from source control (Git) to deployment? Tools? Policies?

Answer

Pipeline security is a layered defence. Each layer adds protection — no single control is sufficient. The goal is to ensure that only reviewed, scanned, approved code reaches production.

Layer 1 — Source Control Security (Azure Repos / GitHub)

- Branch Protection Policies — Require PR reviews (minimum 2 approvers) before merging to main. Reject force pushes. Require passing CI status checks.
- Signed Commits — Require GPG-signed commits to verify the identity of the committer (configures in Azure DevOps branch policies).
- Secret Scanning — Pre-commit hooks using detect-secrets or git-secrets prevent secrets being committed. GitHub Advanced Security or GHAS equivalent scans pushes in real-time.
- CODEOWNERS — Define who must review changes to specific files (e.g., Terraform root modules, pipeline templates, Dockerfiles).

```
# .github/CODEOWNERS or Azure DevOps required reviewers
terraform/prod/**   @platform-team @security-team
azure-pipelines.yml @platform-team
```

Layer 2 — Pipeline Configuration Security

- Protected environments — Mark production environments in Azure DevOps as protected. Only specific pipelines (from specific repos/branches) can deploy to them.
- Pipeline permissions — Restrict which pipelines can use production service connections. Pipelines from feature branches cannot access the prod service connection.
- No self-modifying pipelines — The pipeline YAML file is reviewed before it runs. A pipeline cannot modify itself and re-run with elevated privileges.
- Template enforcement — Use extends: with a required template to ensure all pipelines include mandatory security scanning stages that cannot be removed:

```
# azure-pipelines.yml in application repo
extends:
  template: secure-pipeline-template.yml@templates
  parameters:
    buildContext: src/
    chartPath: charts/myapp
```

Layer 3 — Secret Management

- No plain-text secrets in pipeline YAML — All secrets from Azure Key Vault via linked variable groups.
- Service connections with Workload Identity Federation — No passwords; OIDC-based authentication to Azure subscriptions.
- Least-privilege service connections — Each environment's service connection has only the permissions needed for that environment. Dev connection cannot touch prod resources.
- Secret rotation — Automated 90-day rotation of all pipeline secrets via Key Vault rotation policies.

Layer 4 — Build and Container Security

- Immutable base images — Pin Dockerfile FROM statements to specific digest hashes, not mutable tags.

```
FROM node:20.11.0-alpine3.19@sha256:a1b2c3d4...  # Pinned digest
```

- Image signing — Sign container images with Notation or Cosign. AKS Gatekeeper verifies signatures before admitting pods.
- Distroless / minimal base images — Reduce attack surface by using images with no shell, no package manager.
- Registry access control — Only the CI pipeline can push to the production ACR. Developers can push to the dev ACR only.

Layer 5 — Deployment Security

- Deployment approvals — All production deployments require human approval (see Q8).
- Just-in-time access — Service connections use short-lived tokens. They can't be used outside of pipeline execution context.
- Audit logging — Azure Monitor records all pipeline runs, deployments, and approvals. Alerts on off-hours deployments.
- Post-deployment verification — Automated smoke tests and security scans run against production immediately after deployment.

Layer	Tool / Control
Source control	Branch policies, CODEOWNERS
Secret detection	TruffleHog, detect-secrets
Pipeline config	Protected environments, extends:
Build	Trivy, Checkov, Semgrep
Deployment	Approval gates, OIDC
Runtime	Gatekeeper, Falco

> *Tip: Treat your pipeline configuration as a privileged asset. A malicious change to azure-pipelines.yml can exfiltrate all secrets, bypass all checks, and deploy anything to production. Require the same rigorous review for pipeline files as for production code.*

Q11. What strategies do you use for managing Terraform state in a team? How do you handle state locking and backups in Azure?

Answer

Terraform state management becomes critical in team environments. Incorrect state handling leads to resource conflicts, duplicate resources, or accidental deletions. Azure provides native services that solve every aspect of this.

Remote State Backend — Azure Blob Storage

Store Terraform state in Azure Blob Storage with the azurerm backend. Never store state locally in a team environment:

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstatecompany"
    container_name       = "tfstate"
    key                  = "prod/eastus/terraform.tfstate"
  }
}
```

State Locking — Blob Lease

The azurerm backend uses Azure Blob Storage leases for state locking automatically. When one pipeline runs terraform apply, it acquires a lease on the state blob. Any concurrent run attempting to acquire the lock waits or fails, preventing concurrent state corruption:

```
# Terraform handles this automatically with azurerm backend
# Lock acquired: terraform.tfstate is leased
# Concurrent run sees: Error locking state: state blob is already locked

# Force-unlock if a pipeline crashed leaving a stale lock
terraform force-unlock LOCK_ID
```

State Isolation Strategy

Each environment and region should have its own state file. Never share state between environments:

```
tfstate container:
```

```
dev/eastus/terraform.tfstate
staging/eastus/terraform.tfstate
prod/eastus/terraform.tfstate
prod/westeurope/terraform.tfstate
```

- Use Terraform workspaces OR separate directories + backends for environment isolation. Separate directories (with different backend keys) is generally safer for production — workspace mistakes can affect wrong environments.

State Security

- Enable storage account encryption — Blob Storage encrypts at rest with Microsoft-managed keys (or customer-managed keys for higher compliance).
- Private endpoint on storage account — State files are not accessible from the internet. Only the pipeline's managed identity can access them.
- Storage account access keys disabled — Use Azure AD authentication via the pipeline's managed identity instead of storage account keys.
- RBAC on state container — Only the pipeline's managed identity has Storage Blob Data Contributor on the state container. Developers get Reader at most.

State Backups

- Versioning on Blob Storage — Enable blob versioning on the tfstate container. Azure retains every previous version of the state file. You can restore any previous state with one click.

```
az storage account blob-service-properties update \
  --account-name tfstatecompany \
  --enable-versioning true
```

- Soft delete — Configure 90-day soft delete on blobs so accidentally deleted state can be recovered.
- Before major operations — Take a manual backup of the current state:

```
az storage blob download \
  --account-name tfstatecompany \
  --container-name tfstate \
  --name prod/eastus/terraform.tfstate \
  --file terraform.tfstate.backup.$(date +%Y%m%d)
```

Team Workflow

- Never run terraform apply locally against production state — only the CI/CD pipeline applies to production.
- Developers run terraform plan locally against dev state to validate changes.
- PR review includes terraform plan output — attach plan output as a PR comment using Atlantis or a custom pipeline task.

> *Tip: Use Terragrunt's remote_state configuration to auto-generate the backend configuration per environment. This eliminates copy-pasting backend config across environments and ensures every environment follows the same convention.*

Q12. Explain how to set up remote Terraform backends in Azure (Blob Storage) and enable state locking with Azure CosmosDB or similar.

Answer

The native azurerm Terraform backend uses Azure Blob Storage for state storage AND blob leases for state locking — no CosmosDB is required. CosmosDB is only needed with the older S3-compatible backend pattern or for custom locking implementations. This question addresses both approaches.

Method 1 — Native azurerm Backend (Recommended)

The azurerm backend handles state storage and locking natively using Blob Storage leases. This is the correct, production-grade approach for Azure:

Step 1: Create the Azure Infrastructure for State

```
# Create resource group, storage account, and container
az group create --name rg-terraform-state --location eastus

az storage account create \
  --name tfstatecompanyname \
  --resource-group rg-terraform-state \
  --sku Standard_LRS \
  --https-only true \
  --allow-blob-public-access false \
  --min-tls-version TLS1_2

az storage container create \
```

```
  --name tfstate \
  --account-name tfstatecompanyname

# Enable versioning for state backup
az storage account blob-service-properties update \
  --account-name tfstatecompanyname \
  --enable-versioning true
```

Step 2: Configure Terraform Backend

```
terraform {
  required_version = ">= 1.5"
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstatecompanyname"
    container_name       = "tfstate"
    key                  = "prod/eastus/terraform.tfstate"
    use_azuread_auth     = true   # Use Azure AD, not storage key
  }
}
```

Step 3: Grant Pipeline Identity Access

```
# Grant the pipeline's managed identity access to state
az role assignment create \
  --role 'Storage Blob Data Contributor' \
  --assignee '<managed-identity-object-id>' \
  --scope '/subscriptions/<sub>/resourceGroups/rg-terraform-state'
```

Step 4: Initialize in Azure DevOps Pipeline

```
- task: TerraformTaskV4@4
  inputs:
    command: init
    backendServiceArm: terraform-state-connection
    backendAzureRmResourceGroupName: rg-terraform-state
    backendAzureRmStorageAccountName: tfstatecompanyname
    backendAzureRmContainerName: tfstate
    backendAzureRmKey: prod/eastus/terraform.tfstate
```

How State Locking Works with Blob Leases

When Terraform performs a write operation (apply, destroy), it acquires an exclusive lease on the state blob. The lease has a 15-second automatic expiry with refresh — if the Terraform process dies, the lease automatically expires and another process can acquire it. This is automatic and requires no additional infrastructure (no CosmosDB needed).

Method 2 — CosmosDB Locking (Legacy / Custom Use Cases)

CosmosDB locking was commonly used with the S3 backend pattern (emulated with Azure) before the azurerm backend matured. If required for specific use cases:

```
# Create CosmosDB for locking
az cosmosdb create --name terraform-state-lock --resource-group rg-terraform-state
az cosmosdb sql database create --account-name terraform-state-lock --name TerraformLock
az cosmosdb sql container create \
  --account-name terraform-state-lock \
  --database-name TerraformLock \
  --name locks --partition-key-path /LockID
```

> *Note: The native azurerm backend with Blob lease locking is the recommended approach for Azure. CosmosDB adds unnecessary cost and complexity. Only use CosmosDB locking if you have a specific requirement for a custom backend implementation.*

> *Tip: Use partial backend configuration with -backend-config flags in CI/CD pipelines so the same Terraform code can target different environments without modifying the backend block — pass the storage account name, container, and key as pipeline variables.*

Q13. How do you implement blue-green or canary deployments in AKS using Helm or service mesh (e.g., Istio)?

Answer

AKS supports multiple deployment strategies. Blue-green provides instant rollback capability; canary allows gradual traffic shifting with real-world validation. Both can be implemented with native Kubernetes, Helm, or Istio.

Blue-Green Deployment in AKS

Run two identical deployments (blue = current, green = new). Switch traffic instantly by updating the Kubernetes Service selector.

```
# Blue deployment (currently serving traffic)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-blue
spec:
  replicas: 3
  selector:
    matchLabels: { app: myapp, version: blue }
  template:
    metadata:
      labels: { app: myapp, version: blue }
    spec:
      containers:
      - name: myapp
        image: myacr.azurecr.io/myapp:v1.2.0

# Service — currently pointing to blue
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
    version: blue            # Switch to green for cutover
  ports:
  - port: 80 targetPort: 8080

# Cutover: update selector to green (instant, zero traffic loss)
kubectl patch service myapp-service \
  -p '{"spec":{"selector":{"app":"myapp","version":"green"}}}'

# Rollback: switch back to blue
kubectl patch service myapp-service \
  -p '{"spec":{"selector":{"app":"myapp","version":"blue"}}}'
```

Canary Deployment with Ingress (NGINX / Azure Application Gateway)

Route a percentage of traffic to the new version using NGINX Ingress annotations:

```
# Canary ingress — receives 10% of traffic
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-canary
  annotations:
    nginx.ingress.kubernetes.io/canary: 'true'
    nginx.ingress.kubernetes.io/canary-weight: '10'   # 10% to canary
spec:
  rules:
  - host: myapp.company.com
    http:
      paths:
      - path: /
        backend:
          service: { name: myapp-canary-svc, port: { number: 80 } }
```

Canary with Istio (Fine-Grained Traffic Control)

Istio provides percentage-based traffic splitting at the service mesh level — no need for duplicate ingress rules:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp-vs
spec:
  hosts: [myapp.company.com]
  http:
  - route:
    - destination: { host: myapp-stable, port: { number: 80 } }
      weight: 90
```

```
    - destination: { host: myapp-canary, port: { number: 80 } }
      weight: 10

# Gradually shift: 10 → 25 → 50 → 100 as confidence grows
kubectl patch virtualservice myapp-vs --type=merge \
  -p '{"spec":{"http":[{"route":[{"destination":{"host":"myapp-stable"},"weight":50},
{"destination":{"host":"myapp-canary"},"weight":50}]}]}}'
```

Using Argo Rollouts for Automated Canary

Argo Rollouts automates the canary promotion process based on Prometheus metrics:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
      - setWeight: 10
      - pause: { duration: 10m }     # Monitor for 10 minutes
      - analysis:                    # Auto-check Prometheus metrics
          templates:
          - templateName: error-rate
      - setWeight: 50
      - pause: { duration: 10m }
      - setWeight: 100
```

Method	Traffic Control
Service Selector	100% switch
NGINX Canary	% weight
Istio VirtualService	% weight + headers
Argo Rollouts	Automated + metrics

> *Tip: Combine Argo Rollouts with Prometheus metrics for fully automated canary promotion. Define a success criterion (error rate < 1%, p99 latency < 300ms) and let Argo automatically promote or rollback based on real production metrics.*

Q14. What are liveness and readiness probes? Give a practical example of how incorrect config can lead to downtime.

Answer

Liveness and readiness probes are Kubernetes health check mechanisms that determine whether a container is running correctly and ready to serve traffic. Misconfigurations are one of the most common causes of production outages.

Liveness Probe

Answers: 'Is this container alive?' If the liveness probe fails repeatedly, Kubernetes kills the container and restarts it. Used to detect containers that are deadlocked or stuck in an unrecoverable state.

Readiness Probe

Answers: 'Is this container ready to serve requests?' If the readiness probe fails, Kubernetes removes the pod from the Service endpoints — traffic stops being routed to it, but the container is NOT restarted. Used during startup (app loading config, warming caches) and during degraded states (DB unreachable).

Example Configuration

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30    # Wait 30s before first check
  periodSeconds: 10          # Check every 10s
  failureThreshold: 3        # Kill after 3 consecutive failures
  timeoutSeconds: 5          # Fail if no response within 5s

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
```

```
  initialDelaySeconds: 10    # Start checking 10s after container starts
  periodSeconds: 5           # Check every 5s
  failureThreshold: 3        # Remove from LB after 3 failures
  successThreshold: 1        # One pass restores it to LB
```

Separate Endpoints — Critical Best Practice

The /healthz (liveness) and /ready (readiness) endpoints must be separate and serve different purposes:

- /healthz — Returns 200 always if the process is alive. Never check external dependencies here (DB, cache). If it checks the DB and the DB goes down, Kubernetes will restart all pods unnecessarily, causing a cascading outage.
- /ready — Can check external dependencies. Returns 200 only when the app is ready: DB connection established, config loaded, cache warm. Returns 503 when the app is not ready to serve.

Real-World Downtime Scenario — Incorrect Liveness Probe

Scenario: A Node.js app was deployed with this configuration:

```
# WRONG — Liveness probe checking database connectivity
livenessProbe:
  httpGet:
    path: /health           # This endpoint queries the database
    port: 3000
  initialDelaySeconds: 5   # Too short — app still starting
  failureThreshold: 1      # Kills container on first failure
```

What happened:

- T+0: Database planned maintenance causes a 60-second connection interruption.
- T+5s: /health endpoint returns 500 (DB unreachable). Liveness probe fails on first check (failureThreshold: 1).
- T+5s: Kubernetes kills ALL pods and restarts them simultaneously — no PodDisruptionBudget set.
- T+5-35s: During restart window, ALL pods are in pending/starting state. 100% service outage for 30 seconds.
- T+35s: Pods restart but DB is still down — liveness probe fails again. Kill-restart cycle begins.
- Result: 5-minute complete outage during what should have been a transparent DB maintenance window.

The Correct Configuration

```
livenessProbe:
  httpGet:
    path: /healthz          # NEVER checks DB — always 200 if process alive
    port: 3000
  initialDelaySeconds: 30  # Enough time for app to start
  failureThreshold: 3      # 3 failures = 30 seconds before kill

readinessProbe:
  httpGet:
    path: /ready            # CAN check DB — removes from LB if DB down
    port: 3000
  initialDelaySeconds: 10
  failureThreshold: 3
```

With the correct config, during the DB outage, readiness fails → pods removed from load balancer → users see 503 from ALB (degraded, but no kill loop). When DB recovers, readiness passes → pods re-added → service restored automatically.

Probe	Failure Action
Liveness	Restart container
Readiness	Remove from Service
Startup	Delay liveness checks

> *Tip: Add a startupProbe for applications that take more than 30 seconds to start (JVM apps, large Python apps). The startup probe disables liveness checking until the app has started — preventing premature kills during initialization.*
